

Lab 8: SQL Injection — Extended

From Manual Payloads Through Automated Extraction to OS-Level Compromise

Course: Hands-On Ethical Hacking and Network Defense — Web Application Attacks module companion lab

Estimated time: 4–5 hours — can span two sessions **Format:** Individual; uses your isolated host-only lab (Kali attacking OWASP BWA) **Prerequisite:** Lab 5 (Burp Suite) completed. OWASP BWA running and reachable.

1. Learning Objectives

By the end of this session you will be able to:

- Explain why unsanitized input reaching a SQL query lets an attacker change what that query does
- Bypass a login form with a manually crafted SQL injection payload
- Extract data using UNION-based, error-based, boolean-based blind, and time-based blind injection — and explain when you'd reach for each
- Automate detection and extraction with SQLMap, and escalate a SQL injection into full OS command execution where the database privileges allow it
- Work inside a scope constraint that forbids dumping live data, and still prove a finding
- Recognize and work around a basic input filter
- Demonstrate that a parameterized-query fix actually closes the hole, not just hides the symptom
- Map this entire class of vulnerability to its OWASP Top Ten category and write a fix-oriented finding

2. ⚠ Scope Reminder

Everything below targets a vulnerable application running on your own OWASP BWA VM, on your host-only network. SQL injection is one of the most consequential vulnerability classes in real-world breaches — full customer-database theft, most often, and in the worst cases a path to full server compromise, which this lab deliberately demonstrates in a contained way. That consequence is exactly why it only ever gets practiced here, against a target built to be broken, never against any live system.

3. What You'll Need

- Kali and OWASP BWA running and reachable
- A login-driven application on OWASP BWA with a known SQL injection flaw — **DVWA** is used in the steps below; if your build uses **Mutillidae** instead, the same payloads apply against its login and search fields
- `sqlmap` (pre-installed on Kali)
- Burp Suite, to capture the exact request SQLMap needs (from Lab 5)

4. Procedure

Part A — Why This Works

1. Open DVWA on OWASP BWA and set its security level to **Low** (DVWA Security tab).
2. Before touching the SQL Injection module, think through the mechanism: a web form takes your input and drops it directly into a SQL query string without separating code from data. Write one sentence, in your own words, describing what changes about the query if your input itself contains SQL syntax.
3. Navigate to the **SQL Injection** module and note the single input field it exposes (typically a numeric user ID).

Part B — Manual Injection: Breaking Out of the Query

1. Submit the normal, intended input first (e.g., `1`) and record the expected output.
2. Try a classic breakout payload:

```
1' OR '1'='1
```

3. Record what changed. If the field returns more data than a single ID should, you've confirmed the input is reaching the query unsanitized.
4. Try closing a comment instead of a quote, to see whether the application's exact query structure changes how it reacts:

```
1' OR '1'='1' -- -
```

- Record whether this produced the same result as Step 2 or something different — the difference (or lack of one) tells you something about how the query is built.

Part C — UNION-Based Extraction

- First, find the number of columns the original query returns, by incrementing an `ORDER BY` clause until it errors:

```
1' ORDER BY 1-- -  
1' ORDER BY 2-- -  
1' ORDER BY 3-- -
```

Record the point where it breaks — that tells you the column count.

- Use a `UNION SELECT` with that many columns to pull data the application was never meant to show you:

```
1' UNION SELECT user, password FROM users-- -
```

- Record what came back. If you see usernames and password hashes, you've just extracted authentication data through a field meant to accept a single numeric ID.
- Reflection (1-2 sentences):** You cracked some of these exact password hashes by a completely different route back in Lab 4. Does having two independent paths to the same data change how you'd rate this finding's severity?

Part D — Boolean-Based Blind Injection

Some injectable fields don't display query results directly — they only show a different page state depending on whether your condition was true or false. Simulate that constraint here even though DVWA's Low mode shows output directly, since this technique is what you'll need against less talkative applications.

- Submit a condition you know is true:

```
1' AND '1'='1
```

- Submit a condition you know is false:

```
1' AND '1'='2
```

- Record the difference in the application's response between the two — page content, length, or an error vs. no error. That difference is the entire signal a blind injection attack relies on.
- Write two sentences on why extracting data this way (one true/false question per character) is far slower than Part C's UNION approach, and why an attacker would use it anyway when UNION isn't available.

Part E — Error-Based Injection

When a database is configured to show its errors, you can smuggle data out **inside** the error message itself — often faster than blind techniques and more informative than a bare true/false.

- Try an error-based payload that forces the database to embed a piece of data inside its own error text:

```
1' AND extractvalue(1, concat(0x7e, (SELECT database())))-- -
```

- Record what the error message reveals. If it works, the current database's name appears inside text the application never intended to print.
- Adjust the inner `SELECT` to pull something more specific, such as the current user:

```
1' AND extractvalue(1, concat(0x7e, (SELECT user())))-- -
```

- Reflection (1-2 sentences):** Error-based injection depends on the application actually displaying database errors to the user. What's the simplest possible fix that defeats this specific technique without fixing the underlying injection at all — and why is that fix incomplete?

Part F — Time-Based Blind Injection

When an application shows **no** visible difference at all between a true and a false condition — not even a different error — timing is the only signal left.

1. Submit a payload that delays the response only if a condition is true:

```
1' AND IF(1=1, SLEEP(5), 0)-- -
```

2. Time the response. Record whether it took approximately five seconds longer than normal.
3. Submit the false version and confirm the response returns immediately:

```
1' AND IF(1=2, SLEEP(5), 0)-- -
```

4. Record the difference. This is the last resort of manual injection — no data, no errors, just a stopwatch — and it's also the slowest, since every single bit of information costs a multi-second wait.

Part G — Automating with SQLMap: Database Enumeration

1. In Burp Suite, intercept one request from the SQL Injection module (your Part A/B traffic) and save it as a raw request file (**Save item** in Burp, or copy the raw request into a text file named `request.txt`).
2. Point SQLMap at the captured request:

```
sqlmap -r request.txt --batch --dbs
```

3. Record every database SQLMap lists.
4. Enumerate tables in the application's own database:

```
sqlmap -r request.txt --batch -D dvwa --tables
```

5. Dump the `users` table:

```
sqlmap -r request.txt --batch -D dvwa -T users --dump
```

6. Record how much faster and more complete this was than Parts B–F combined, and what SQLMap did automatically that you were doing by hand (column counting, blind-technique fallback, hash format detection).

Part H — Escalating Impact: From SQL Injection to OS Shell

Some database configurations let a sufficiently privileged database account write files or execute commands on the underlying host — turning a SQL injection into full server access, the same category of outcome as Lab 3's exploit, reached through a completely different door.

1. Ask SQLMap to check for and attempt an OS-level shell through the injection point:

```
sqlmap -r request.txt --batch --os-shell
```

2. If SQLMap succeeds, run `id` and `whoami` from the resulting shell and record the access level — compare it directly against what you got in Lab 3.
3. If it fails (the database account may not have the required `FILE` privilege), record that outcome too — a failed privilege-escalation attempt, properly documented, is itself a valid and common finding.
4. Whatever the outcome, do **not** create files, users, or any persistence mechanism — same Phase 3 discipline as Lab 3.
5. **Reflection (2-3 sentences):** You've now reached "root on Metasploitable" through two unrelated vulnerabilities — an FTP backdoor (Lab 3) and a web input field (this lab). What does that say about treating any single vulnerability as the whole picture of a system's risk?

Part I — Scenario: A Second Target Under a Tighter Scope

A new, fictional instruction arrives mid-engagement: "We've confirmed the vulnerability class is real across our applications. For the payment-processing module specifically, you may enumerate structure only — no data dumps, under any circumstance, including test data. We don't want a repeat of a past incident where a 'test' export left real records in a consultant's report."

1. Pick a **different** injectable field on OWASP BWA than the one you've used so far — DVWA's **SQL Injection (Blind)** module, or a search/comment field in Mutillidae, work well.

2. Confirm injectability using only the lightest possible technique from Parts B-F — don't reach for SQLMap's `--dump` at all.
3. Using SQLMap, enumerate **structure only** — databases and table names — and stop there deliberately:

```
sqlmap -r request2.txt --batch --dbs
sqlmap -r request2.txt --batch -D <database> --tables
```

4. Write the finding as the scope requires: enough evidence to prove the injection is real and serious, with **zero** actual data included, not even redacted.
5. **Reflection (1-2 sentences):** Is a report with no extracted data a weaker finding than Part G's, or does Part G's dump plus this scope instruction tell you something about how that extra evidence should have been handled in the first place?

Part J — Basic Filter Evasion

Some applications attempt simple input filtering rather than fixing the underlying query. DVWA's Medium level (used in Part K below) is a mild example — here, practice the general technique on your Part B payload before Part K applies it against an actual filter.

1. If a naive filter blocks the literal string `OR`, try breaking it up with inline comments, which some database engines still parse correctly:

```
1' /*comment*/OR/*comment*/'1'='1
```

2. If a filter is case-sensitive and only blocks lowercase `union select`, try mixed case:

```
1' UnIoN SeLeCt user, password FROM users-- -
```

3. Record whether either technique would plausibly bypass a naive, pattern-matching filter — you don't need DVWA to actually block Low-level input to reason through this; treat it as a thought exercise you'll verify for real in Part K.

Part K — Confirming the Fix

1. In DVWA, raise the security level to **Medium**, then **High**.
2. Re-attempt your Part B payload (`1' OR '1'='1`) at each level, and if it's blocked, try one of your Part J evasion ideas against it.
3. Record at which level the payload stops working **even with evasion attempts**, and — if you can see DVWA's source for each level (**View Source** button, if enabled) — what changed in the underlying query (look specifically for `mysqli_real_escape_string` or a parameterized/prepared statement).
4. **Reflection (2-3 sentences):** Medium-level DVWA often just escapes special characters rather than fully parameterizing the query. Is escaping input an adequate fix, or a partial one? What would you tell a developer who proposed escaping as their only fix?

Part L — Report the Finding

Produce a single finding write-up, addressed to the application's developer, containing:

1. The vulnerable parameter(s) and the payloads that proved each technique (UNION, error-based, boolean-blind, time-based — note which worked where)
2. What you were able to extract in Part G, and what you deliberately did **not** extract in Part I under the tighter scope — explain the difference in one sentence
3. Your Part H escalation result (success or documented failure)
4. The OWASP Top Ten category this belongs to
5. The fix: name parameterized queries/prepared statements specifically, not just "sanitize input" or "add a filter" — and explain in one sentence why Part J and K showed those weaker fixes to be insufficient

5. What to Submit

1. Part A — your one-sentence mechanism explanation
2. Part B — your breakout results
3. Part C — your column count, extracted data, and reflection
4. Part D — your true/false comparison and reflection
5. Part E — your error-based extraction results and reflection
6. Part F — your timing results

7. Part G — your SQLMap database, table, and dump output
8. Part H — your `--os-shell` result (success or documented failure) and reflection
9. Part I — your structure-only enumeration and reflection
10. Part J — your evasion reasoning
11. Part K — the security level where your payload (and evasion attempts) stopped working, and your reflection
12. Part L — the full finding write-up

6. Troubleshooting

Symptom	Likely cause / fix
<code>sqlmap</code> reports the parameter isn't injectable	Confirm your captured request includes the session cookie — DVWA requires authentication, and SQLMap needs that cookie in the request file to reach the vulnerable page at all
UNION payload returns a column-count error	Your <code>ORDER BY</code> test in Part C, Step 1 likely stopped one column too early or late — recheck exactly where the error first appeared
Error-based payload returns no error at all	The application may be suppressing database errors from display — fall back to boolean-based or time-based blind technique instead
<code>--os-shell</code> fails immediately	Common and expected — it requires the database account to hold <code>FILE</code> privilege and the web root to be writable; document the failure as your Part H result rather than troubleshooting further
DVWA's SQL Injection module isn't visible	Confirm you're logged in and the DVWA security level is set (Low, to start) — some builds hide modules behind a setup/database-init step on first use
SQLMap dump shows hashes, not plaintext passwords	Expected — SQLMap extracts what the database actually stores; cracking the hashes is Lab 4's job, not this one

This is Lab 8 in the course sequence. It builds directly on Lab 5's Burp Suite skills and Lab 7's vulnerability-triage mindset — Part I's scope constraint and Part H's severity-escalation finding are the same judgment calls Lab 7 introduced, now applied to a web-layer vulnerability instead of a network-layer scan result.